Steel Plant Energy Consumption Forecasting

Methodology Summary • 15-minute-ahead regression

Objective & dataset

Predict **next_interval_Usage_kWh** using current plant telemetry and previously observed electricity usage. The provided training data contains **28,031 observations, 11 feature columns and one target**. Measurements describe current consumption, reactive power, CO2, power factors, time of day, weekday and operational load. The objective is low RMSE and MAE on a subsequent chronological period.

Exploratory analysis & data quality

- The training data contained **no missing values or exact duplicate rows** in the initial audit.
- Electricity usage is **strongly right-skewed** (mean approximately **27.93 kWh**; median approximately **4.61 kWh**), making large errors during high-demand periods particularly important for RMSE.
- The raw date field contains midnight labeling irregularities, so we preserve the supplied observation order for temporal splitting and use **NSM to encode time of day**.

Feature engineering & modeling

Our model combines present operating conditions with historical consumption patterns. Original inputs include current Usage_kWh, lagging and leading reactive power, CO2, both power factors, NSM, weekday/weekend indicators and load type. Calendar and cyclical time-of-day features help represent recurring operating patterns without treating midnight as a discontinuity.

Causal usage lags are constructed at **1, 2, 3, 4, 8, 12, 24, 48, 96, 192 and 672 intervals**. Rolling usage means and standard deviations over **4, 8, 24 and 96 intervals** summarize recent demand levels and variability; a one-step usage-change feature captures short-term movement. Each feature uses only measurements available at or before the prediction timestamp, never the future target.

We compared **LightGBM, XGBoost, CatBoost and HistGradientBoosting**. The selected **LightGBM** configuration uses **350 estimators, 20 leaves, learning rate 0.035 and L2 regularization 5**. This model was selected on chronological validation RMSE, with MAE also recorded.

Validation design

We retained the original observation sequence and used a **70% training / 15% validation / 15% untouched final holdout** division, rather than random shuffling. Because each row predicts the following 15-minute interval, a boundary observation was excluded where its target would refer to the first observation of the next split. Candidate models were compared on the same validation period using RMSE and MAE. After selecting the lowest-validation-RMSE configuration, it was refitted on the pre-holdout data and evaluated once on the final holdout. The final serialized model was subsequently trained using the available training data.

Measured results

Model / evaluation	RMSE (kWh)	MAE (kWh)
LightGBM — validation	8.5772	4.0902
XGBoost — validation	8.6542	4.1119
HistGradientBoosting 200 — validation	8.6584	4.1508
HistGradientBoosting 350 — validation	8.6728	4.1307
CatBoost — validation	8.8226	4.3748
Selected LightGBM — holdout	8.7751	4.0793
Persistence baseline — holdout	12.5233	5.5566

Holdout RMSE improves by 29.9% over the persistence baseline.

Industrial implications & interpretability

The selected model reduces holdout RMSE by approximately **29.9%** relative to predicting that the next interval will equal the current usage. Reliable short-horizon forecasts could help operators anticipate demand spikes, plan discretionary loads and investigate unusually high consumption. Current and historical usage, power factors and reactive-power measurements are plausible predictors, but their association with predictions should not be interpreted as causation. Actual financial savings would require tariff schedules and operational intervention data. Abrupt plant changes and model drift remain limitations.

Reproducibility & inference

The GitHub project includes the training script, **energy_forecasting_model.ipynb**, dependency list, saved **LightGBM model**, evaluation reports and an inference function **predict(test_df)**. Inference accepts the **11 input feature columns without the target**, constructs historical lag/rolling features from observed data, and returns predictions **without retraining**. For contiguous hidden-test data, saved training-tail history can initialize early lag windows.